MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name Poonam Sharma

Subject Name with Code Deep Learning & 7CS4-01

Semester 7semester **Session** 2025-26 (Odd)

Unit No. Unit 1 **Submission Date** 23-Aug-25

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Introduction of artificial neural network and deep learning	1 - 6	6
2	Characteristrics of neural network terminology	7-8	2
3	Neurons, Preceptron, Back propagation	9-12	4
4	Basics learning low of Back propagation	13-16	4
5	Activation and loss function	17-20	4
6	Function approximation	21-22	2
7	Application of Activation and loss function	23-24	2

Poonam
Faculty

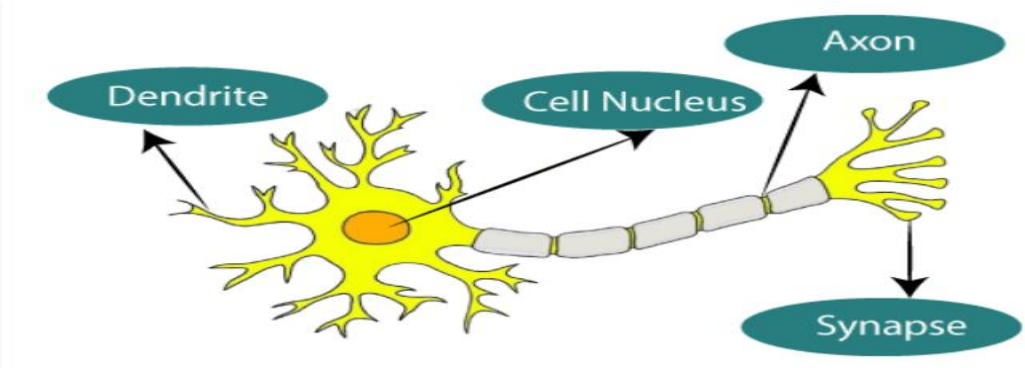
HOD

25/08/2025
Dean Academics

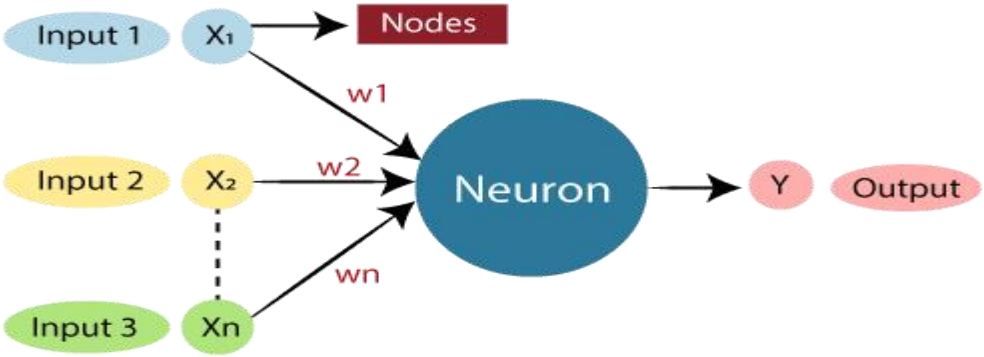
UNIT-1
Introduction to Neural Networks
Lecture No.1

What is Artificial Neural Network?

The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain. Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.



The typical Artificial Neural Network looks something like the given figure.



Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.

Relationship between Biological neural network and artificial neural network:

Biological Neural Network	Artificial Neural Network

Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner. The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.

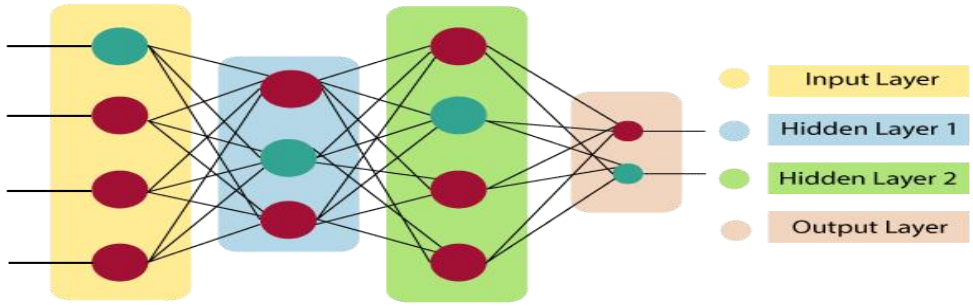
There are around 1000 billion neurons in the human brain. Each neuron has an association point somewhere in the range of 1,000 and 100,000. In the human brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly. We can say that the human brain is made up of incredibly amazing parallel processors.

We can understand the artificial neural network with an example, consider an example of a digital logic gate that takes an input and gives an output. "OR" gate, which takes two inputs. If one or both the inputs are "On," then we get "On" in output. If both the inputs are "Off," then we get "Off" in output. Here the output depends upon input. Our brain does not perform the same task. The outputs to inputs relationship keep changing because of the neurons in our brain, which are "learning."

Architecture of an artificial neural network

To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers. Lets us look at various types of layers available in an artificial neural network.

Artificial Neural Network primarily consists of three layers:



1. Input Layer:

As the name suggests, it accepts inputs in several different formats provided by the programmer.

2. Hidden Layer:

The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

3. Output Layer:

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer. The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^{n} Wi * Xi + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Advantages of Artificial Neural Network (ANN)

1. Parallel processing capability:

Artificial neural networks have a numerical value that can perform more than one task simultaneously.

2. Storing data on the entire network:

Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

3. Capability to work with incomplete knowledge:

After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

4. Having a memory distribution:

For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network. The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

5. Having fault tolerance:

Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

Disadvantages of Artificial Neural Network:

1. Assurance of proper network structure:

There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.

2. Unrecognized behavior of the network:

It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.

3. Hardware dependence:

Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.

4. Difficulty of showing the issue to the network:

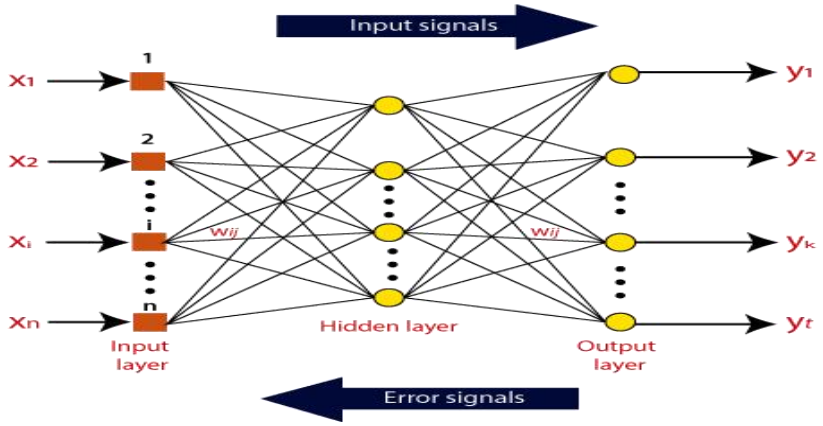
ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.

5. The duration of the network is unknown:

The network is reduced to a specific value of the error, and this value does not give us optimum results.

How do artificial neural networks work?

Artificial Neural Network can be best represented as a weighted directed graph, where the artificial neurons form the nodes. The association between the neurons outputs and neuron inputs can be viewed as the directed edges with weights. The Artificial Neural Network receives the input signal from the external source in the form of a pattern and image in the form of a vector. These inputs are then mathematically assigned by the notations x(n) for every n number of inputs.



Afterward, each of the input is multiplied by its corresponding weights (these weights are the details utilized by the artificial neural networks to solve a specific problem). In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network. All the weighted inputs are summarized inside the computing unit.

If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response. Bias has the same input, and weight equals to 1. Here the total of weighted inputs can be in the range of 0 to positive infinity. Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

The activation function refers to the set of transfer functions used to achieve the desired output. There is a different kind of the activation function, but primarily either linear or non-linear sets of functions. Some of the commonly used sets of activation functions are the Binary, linear, and Tan hyperbolic sigmoidal activation functions. Let us take a look at each of them in details:

Binary:

In binary activation function, the output is either a one or a 0. Here, to accomplish this, there is a threshold value set up. If the net weighted input of neurons is more than 1, then the final output of the activation function is returned as one or else the output is returned as 0.

Sigmoidal Hyperbolic:

The Sigmoidal Hyperbola function is generally seen as an "**S**" shaped curve. Here the tan hyperbolic function is used to approximate output from the actual net input. The function is defined as:

F(x) = (1/1 + exp(-????x))

Where ???? is considered the Steepness parameter.

References:

https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/

UNIT-1(Introduction to Neural Networks)
Deep Learning
Lecture No.2

What is Deep Learning?

Deep learning is the branch of machine learning which is based on artificial neural network architecture. An artificial neural network or ANN uses layers of interconnected nodes called neurons that work together to process and learn from the input data.

In a fully connected Deep neural network, there is an input layer and one or more hidden layers connected one after the other. Each neuron receives input from the previous layer neurons or the input layer. The output of one neuron becomes the input to other neurons in the next layer of the network, and this process continues until the final layer produces the output of the network. The layers of the neural network transform the input data through a series of nonlinear transformations, allowing the network to learn complex representations of the input data.

Today Deep learning has become one of the most popular and visible areas of machine learning, due to its success in a variety of applications, such as computer vision, natural language processing, and Reinforcement learning.

Deep learning can be used for supervised, unsupervised as well as reinforcement machine learning. It uses a variety of ways to process these:

- **Supervised Machine Learning:** Supervised machine learning is the machine learning technique in which the neural network learns to make predictions or classify data based on the labeled datasets. Here we input both input features along with the target variables. the neural network learns to make predictions based on the cost or error that comes from the difference between the predicted and the actual target, this process is known as backpropagation. Deep learning algorithms like Convolutional neural networks, Recurrent neural networks are used for many supervised tasks like image classifications and recognization, sentiment analysis, language translations, etc.

- **Unsupervised Machine Learning:** Unsupervised machine learning is the machine learning technique in which the neural network learns to discover the patterns or to cluster the dataset based on unlabeled datasets. Here there are no target variables. while the machine has to self-determined the hidden patterns or relationships within the datasets. Deep learning algorithms like autoencoders and generative models are used for unsupervised tasks like clustering, dimensionality reduction, and anomaly detection.

- **Reinforcement Machine Learning**: Reinforcement Machine Learning is the machine learning technique in which an agent learns to make decisions in an environment to maximize a reward signal. The agent interacts with the environment by taking action and observing the resulting rewards. Deep learning can be used to learn policies, or a set of actions, that maximizes the cumulative reward over time. Deep reinforcement learning algorithms like Deep Q networks and Deep Deterministic Policy Gradient (DDPG) are used to reinforce tasks like robotics and game playing etc.

Characteristics of ANN

- It is neurally implemented mathematical model
- It contains huge number of interconnected processing elements called neurons to do all operations
- Information stored in the neurons are basically the weighted linkage of neurons
- The input signals arrive at the processing elements through connections and connecting weights.
- It has the ability to learn, recall and generalize from the given data by suitable assignment and adjustment of weights.
- The collective behavior of the neurons describes its computational power, and no single neuron carries specific information.

References:

https://www.sciencedirect.com/topics/computer-science/deep-neural-network#:~:text=Like%20the%20human%20brain%2C%20a,called%20a%20deep%20neural%20network.

UNIT-1(Introduction to Neural Networks)
Perceptron
Lecture No.3

Perceptron model in Machine Learning

Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, **Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence**.

Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

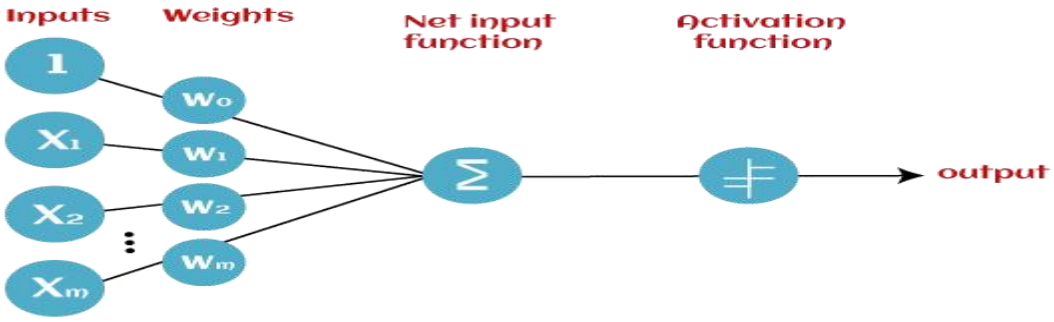
What is Binary classifier in Machine Learning?

In Machine Learning, binary classifiers are defined as the function that helps in deciding whether input data can be represented as vectors of numbers and belongs to some specific class.

Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a **classification algorithm that can predict linear predictor function in terms of weight and feature vectors.**

Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:



Simple Perceptron Layer

- **Input Nodes or Input Layer:**

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

- **Wight and Bias:**

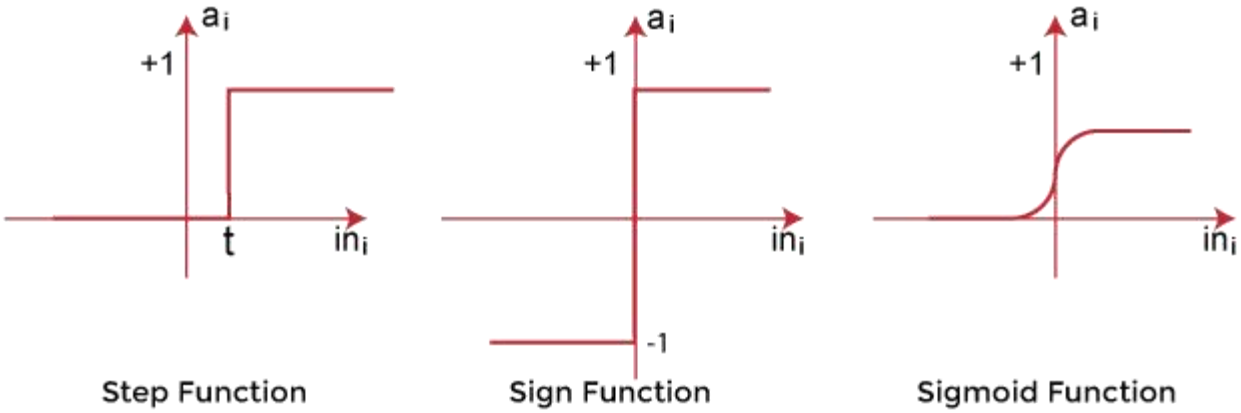
Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

- **Activation Function:**

These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

Types of Activation functions:

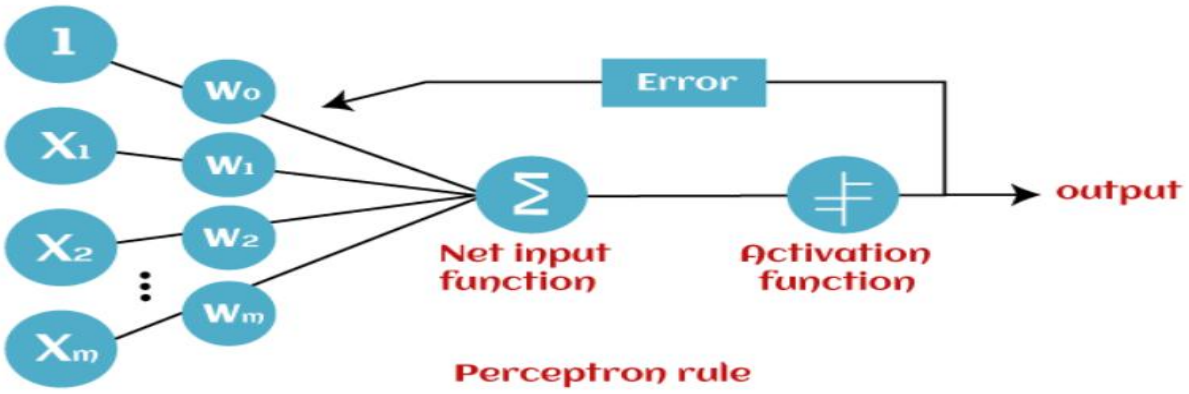
- Sign function
- Step function, and
- Sigmoid function



The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.

How does Perceptron work?

In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the **step function** and is represented by **'f'**.



This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

Perceptron model works in two important steps as follows:

Step-1

In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

∑wi*xi = x1*w1 + x2*w2 +…wn*xn

Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.

∑wi*xi + b

Step-2

In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

Y = f(∑wi*xi + b)

Characteristics of Perceptron

The perceptron model has the following characteristics.

- Perceptron is a machine learning algorithm for supervised learning of binary classifiers.
- In Perceptron, the weight coefficient is automatically learned.
- Initially, weights are multiplied with input features, and the decision is made whether the neuron is fired or not.
- The activation function applies a step rule to check whether the weight function is greater than zero.

- The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
- If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

Limitations of Perceptron Model

A perceptron model has limitations as follows:

- The output of a perceptron can only be a binary number (0 or 1) due to the hard limit transfer function.
- Perceptron can only be used to classify the linearly separable sets of input vectors. If input vectors are non-linear, it is not easy to classify them properly.

References:

https://www.geeksforgeeks.org/what-is-perceptron-the-simplest-artificial-neural-network/

UNIT-1(Introduction to Neural Networks)

Back propagation
Lecture No.4

What is Back propagation?

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Features of Backpropagation

1. It is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
2. It is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. Training is done in the three stages :
   - the feed-forward of input training pattern
   - the calculation and backpropagation of the error
   - updation of the weight.

Working of Backpropagation

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

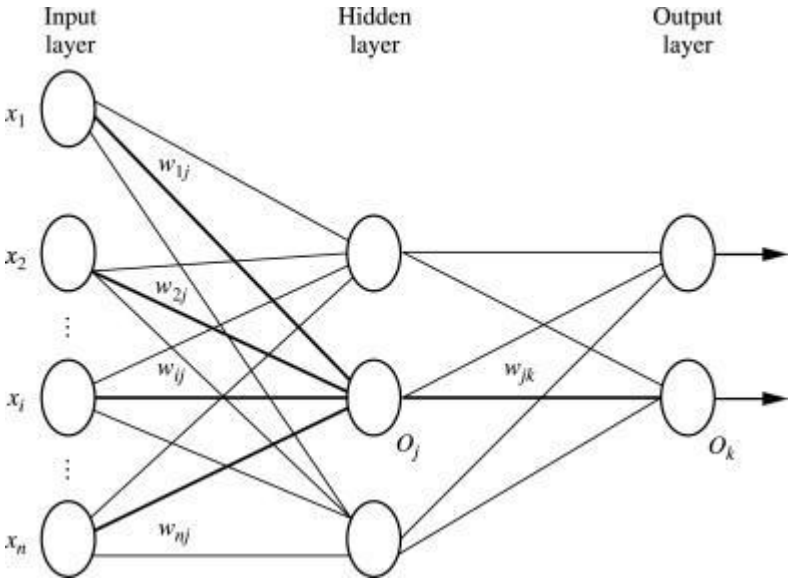
Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs ,Backpropagation

Error= Actual Output – Desired Output

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Parameters

- x = inputs training vector $x=(x_1,x_2,\ldots\ldots\ldots x_n)$.
- t = target vector $t=(t_1,t_2\ldots\ldots\ldots\ldots t_{n)}$.
- δ_k = error at output unit.
- δ_j = error at hidden layer.
- α = learning rate.
- v_{0j} = bias of hidden unit j.

Training Algorithm

Step 1: Initialize weight to small random values.

Step 2: While the steps stopping condition is to be false do step 3 to 10.

Step 3: For each training pair do step 4 to 9 (Feed-Forward).

Step 4: Each input unit receives the signal unit and transmits the signal x_i signal to all the units.

Step 5 : Each hidden unit Zj (z=1 to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \Sigma x_i v_{ij} \quad (i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signals to all units in the layer about i.e output units

For each output l=unit y_k = (k=1 to m) sums its weighted input signals.

$$y_{ink} = w_{0k} + \Sigma z_i w_{jk} \quad (j=1 \text{ to } a)$$

and applies its activation function to calculate the output signals.

$$y_k = f(y_{ink})$$

Backpropagation Error:

Step 6: Each output unit y_k (k=1 to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = (t_k - y_k) + y_{ink}$$

Step 7: Each hidden unit Z_j (j=1 to a) sums its input from all units in the layer above

$$\delta_{inj} = \Sigma \delta_j w_{jk}$$

The error information term is calculated as :

$$\delta_j = \delta_{inj} + z_{inj}$$

Updation of weight and bias

Step 8: Each output unit y_k (k=1 to m) updates its bias and weight (j=1 to a). The weight correction term is given by :

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(new)} = w_{jk(old)} + \Delta w_{jk}$

$$w_{0k(new)} = w_{ok(old)} + \Delta w_{ok}$$

for each hidden unit z_j (j=1 to a) update its bias and weights (i=0 to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

$$\Delta v_{0j} = \alpha \delta_j$$

Therefore $v_{ij(new)} = v_{ij(old)} + \Delta v_{ij}$

$$v_{0j(new)} = v_{0j(old)} + \Delta v_{0j}$$

Step 9: Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

Need for Backpropagation:

Backpropagation is "backpropagation of errors" and is very useful for training neural networks. It's fast, easy to implement, and simple. Backpropagation does not require any parameters to be set, except the number of inputs. Backpropagation is a flexible method because no prior knowledge of the network is required.

Types of Backpropagation

There are two types of backpropagation networks.

- **Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- **Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

Advantages:

- It is simple, fast, and easy to program.
- Only numbers of the input are tuned, not any other parameter.
- It is Flexible and efficient.
- No need for users to learn any special functions.

Disadvantages:

- It is sensitive to noisy data and irregularities. Noisy data can lead to inaccurate results.
- Performance is highly dependent on input data.
- Spending too much time training.
- The matrix-based approach is preferred over a mini-batch.

References:

https://www.geeksforgeeks.org/backpropagation-in-neural-network/

UNIT-1(Introduction to Neural Networks)

Basic Learning Laws

Lecture No.5

Types of Learning Rules in ANN

ANN or artificial neural networks are the computing systems developed by taking inspiration from the biological neural networks; the human brain being its major unit. These neural networks are made functional with the help of training that follows some kind of a learning rule.

A learning rule in ANN is nothing but a set of instructions or a mathematical formula that helps in reinforcing a pattern, thus improving the efficiency of a neural network. There are 6 such learning rules that are widely used by neural networks for training.

1. Hebbian Learning Rule

Developed by Donald Hebb in 1949, Hebian learning is an unsupervised learning rule that works by adjusting the weights between two neurons in proportion to the product of their activation.

According to this rule, the weight between two neurons is decreased if they are working in opposite directions and vice-versa. However, if there is no correlation between the signals, then the weight remains the same. As for the sign a weight carries, it is positive when both the nodes are either positive or negative. However, in case of a one node being either positive or negative, the weight carries a negative sign.

Formula:

$\Delta w = \alpha x i y$

where, Δw is the change in weight

- α is the learning rate
- xi is the input vector and,
- y is the output

2. Perceptron Learning Rule

Developed by Rosenblatt, the perceptron learning rule is an error correction rule, used in a single layer feed forward network. Like Hebbian learning, this also, is a supervised learning rule.

This rule works by finding the difference between the actual and desired outputs and adjusts the weights according to that. Naturally, the rule requires the input of a set of vectors, along with the weights so that it can produce an output.

Formula:
$w = w + \eta(y - \hat{y})x$

- w is the weight
- η is the learning rate
- x is the input vector
- y is the actual label of the input vector
- ŷ is the predicted label of the input vector

3. Correlation Learning Rule

With a similar principle as the Hebbian rule, the correction rule also increases or decreases the weights based on the phases of the two neurons.

If the neurons are in the opposite phase to each other, the weight should be towards the negative side and if they are in the same phase to each other, the weight should be towards the positive side. The only thing that makes this rule different from the Hebbian learning rule is that it is supervised in nature.

Formula:
$\Delta w = \alpha x_i t_j$

- Δw is the change in weight
- α is the learning rate
- xi is the input vector and,
- tj is the target value

4. Competitive Learning Rule

The competitive learning rule is unsupervised in nature and as the name says, is based on the principle of competition amongst the nodes. That is why it is also known as the 'Winner takes it all' rule.

In this rule, all the output nodes represent the input pattern and the best one, meaning, the one having the most number of outputs is the winner. This winner node is then assigned the value 1 and the others that lose, remain at 0. Naturally, only one neuron remains active at once.

Formula:
$\Delta w_{ij} = \eta * (x_i - w_{ij})$

where,

- Δw_{ij} is the changes in weight between the ith input neuron and jth output neuron
- η is the learning rate
- x_i is the input vector
- w_{ij} is the weight between the ith input neuron and jth output neuron

5. Delta Learning Rule

Developed by Bernard Widrow and Marcian Hoff, Delta learning rule is a supervised learning rule with a continuous activation function.

The main aim of this rule is to minimize error in the training patterns and thus, it is also known as the least mean square method. The principle used here is that of an infinite gradient descent and the changes in weight is equal to the product of the error and the input.

Formula:
$\Delta w_{ij} = \eta * (d_j - y_j) * f'(h_j) * x_i$

where, Δw_{ij} is the changes in weight between the ith input neuron and jth output neuron

- η is the learning rate
- d_j is the target output
- y_j is the actual output
- $f'(h_j)$ is the derivative of the activation function
- x_i is the ith input

6. Out Star Learning Rule

Developed by Grossberg, Out Star learning is a supervised learning rule that works with a pair of nodes arranged in a network layer.

According to this rule, the weights linked to a given node are equal to target output along that particular link. Also, there is a single output layer of neurons, where each neuron is a cluster of input vectors. This algorithm is mostly used for patterns and clustering related tasks.

Formula

$\Delta w_{ij} = \eta * (x_i - w_j) * f(w_j)$

where,

- Δw_{ij} is the change in weight
- η is the learning rate
- x_i is the input pattern
- w_j is the weight vector
- $f(w_j)$ is the activation function

References:

https://www.tutorialspoint.com/types-of-learning-rules-in-ann

UNIT-1(Introduction to Neural Networks)
Activation and Loss function
Lecture No.6

Activation Functions

Activation functions serve as the building blocks of deep neural networks. They are applied to the weighted sum of inputs to introduce non-linearity, which enables neural networks to learn complex relationships in data. There are several types of activation functions, each with its own characteristics:

- **Sigmoid**: The sigmoid function maps input values to the range (0, 1), making it suitable for binary classification tasks. However, it suffers from the vanishing gradient problem, which can slow down training.
- **Tanh (Hyperbolic Tangent):** Tanh is similar to the sigmoid but maps input values to (-1, 1), providing a better symmetry around zero. It also suffers from the vanishing gradient problem.
- **ReLU (Rectified Linear Unit):** ReLU is one of the most widely used activation functions. It is computationally efficient and overcomes the vanishing gradient problem. However, it is susceptible to the dying ReLU problem, where neurons can become inactive during training.
- **Leaky ReLU:** Leaky ReLU addresses the dying ReLU problem by allowing a small gradient for negative input values, preventing neurons from becoming entirely inactive.
- **Parametric ReLU (PReLU):** PReLU takes the idea of Leaky ReLU a step further by allowing the gradient to be learned during training, rather than being a fixed parameter.
- **Swish**: Swish is a smooth and differentiable activation function that has gained popularity due to its improved performance in some scenarios.

Loss Functions

Loss functions, also known as cost functions or objective functions, measure the difference between the predicted outputs of a model and the actual target values. The choice of the loss function depends on the type of task a neural network is designed for:

- **Mean Squared Error (MSE):** MSE is commonly used for regression tasks. It computes the average squared difference between predicted and actual values. The goal is to minimize this error.

- **Cross-Entropy Loss:** Cross-entropy loss is widely used for classification tasks. It measures the dissimilarity between predicted class probabilities and true class labels. Cross-entropy encourages the model to assign higher probabilities to the correct class.
- **Hinge Loss:** Hinge loss is often used for support vector machines (SVMs) and some types of binary classification problems. It aims to maximize the margin between classes.
- **Categorical Cross-Entropy:** Similar to cross-entropy loss, this variant is used for multi-class classification problems. It measures the divergence between predicted class probabilities and one-hot encoded target labels.

References:

https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks

UNIT-1(Introduction to Neural Networks)
Function Approximation &Applications
Lecture No.7

Function Approximation

Function approximation is a key component of reinforcement learning (RL) that enables agents to generalize their knowledge and make wise judgements in scenarios not explicitly encountered in training.

Strengthening When an agent interacts with its surroundings, it learns the best ways to optimize cumulative rewards over a period of time. The state and action spaces in many real-world settings can be large and continuous, making it impractical to store and process all possible combinations computationally. This problem is addressed by function approximation, which enables agents to make defensible decisions without doing a thorough investigation by modelling and approximating the value or policy functions.

The Need for Function Approximation

Maintaining explicit representations of value functions or policies is impractical in many real-life learning situations. Due to the large and continuous state and action spaces. The curse of dimensionality poses a serious challenge, resulting in higher memory and processing demands. This problem is addressed by function approximation, which allows agents to make sensible judgements in unknown conditions by generalizing previously learned information.

Types of Function Approximation in RL

1. Linear Function Approximation

A straightforward yet effective method for approximating value or policy functions is the linear function approximation, which uses a linear combination of features. Training teaches the weights associated with these qualities, which indicate pertinent parts of the state. The linear function approximation, although straightforward, has proven effective in a variety of reinforcement learning situations, including as control and prediction tasks.

2. Polynomial Function Approximation

By adding higher-order elements, polynomial function approximation expands on the concept of linear approximation. As a result, nonlinear interactions between state characteristics and values can be captured by the model. The curse of dimensionality can

make polynomial function approximation difficult in high-dimensional domains, even though it is more expressive than linear approximation.

3. Neural Networks in Function Approximation

In reinforcement learning, neural networks have become a widely used and potent technique for function approximation. Deep Reinforcement Learning (DRL), which makes use of neural networks to estimate complex and high-dimensional functions, blends reinforcement learning (RL) with deep learning approaches. Neural networks are used in reinforcement learning (RL) for function approximation in techniques such as Deep Q Networks (DQN) and Policy Gradient approaches.

Applications of Function Approximation in RL

Applications for function approximation are widely used in many different fields. Among the noteworthy instances are:

1. Robotics

Function approximation helps agents learn sophisticated motor skills and control policies in continuous state and action spaces in robotic control problems. Particularly effective in applications ranging from movement to robotic manipulation are neural networks.

2. Finance

RL with function approximation is used in finance for algorithmic trading, risk management, and portfolio optimization. In financial applications, the capacity to generalize techniques to a variety of market circumstances is essential.

3. Healthcare

Applications in healthcare require decision-making in fluid and unpredictable situations. In healthcare contexts, function approximation helps with disease prognosis, resource allocation, and customized treatment planning.

References:

Deep Learning, Ian Goodfellow Yoshua Bengio Aaron Courville, MIT Press, 2017